

二次开发

RIG-Hover 开源文档与开发教程

本文面向希望快速熟悉、二次开发和调试 RIG-Hover 的嵌入式开发者。项目来源为 [LuwuDynamics/rig_omni](https://github.com/LuwuDynamics/rig_omni)，RIG-Hover 是该仓库中基于 ESP32-S3 的双轮自平衡机器人形态。

1. 项目定位

RIG-Omni 是一套统一的 ESP32-S3 机器人固件框架，用同一套应用层支持不同机器人形态。当前代码中主要包含：

- RIG-Puppy：5 舵机机器狗
- RIG-Hover：1 个头部舵机 + 2 个串口 FOC 轮电机的双轮自平衡机器人

RIG-Hover 的价值不只是“能跑起来”，而是把语音交互、表情显示、Wi-Fi 配网、MCP 远程控制、IMU 姿态采集、双轮平衡控制和 Web 调参放在同一个 ESP32-S3 固件工程里。对开发者来说，它是一个很好的“智能机器人固件骨架”。

2. RIG-Hover 硬件组成

模块	代码入口	说明
主控	ESP32-S3	运行 ESP-IDF、FreeRTOS、Wi-Fi、音频、显示和运动控制
屏幕	GC9A01 240x240 圆屏	使用 SPI + LVGL + EAF 表情动画
姿态传感器	QMI8658C IMU	I2C 读取 roll、pitch、yaw，用于平衡控制
头部舵机	XGO 串口协议，ID 3	控制头部左右转动
左右轮电机	串口 FOC 电机，ID 1/2	差速驱动与自平衡力矩输出
音频	I2S Mic/Speaker	语音唤醒、ASR/TTS、提示音
摄像头	ESP32 camera DVP	可选拍照工具

模块	代码入口	说明
按键/触摸	Boot GPIO0 + Touch GPIO3	配网、聊天开关、长按清除 NVS

RIG-Hover 专属引脚集中在 `main/boards/hover/board_config.h`：

```
#define XGO_UART_TX_PIN GPIO_NUM_46
#define XGO_UART_RX_PIN GPIO_NUM_38
#define TOUCH_BUTTON_GPIO GPIO_NUM_3
#define IMU_I2C_SDA GPIO_NUM_14
#define IMU_I2C_SCL GPIO_NUM_48
```

3. 代码结构速览

重点目录如下：

```
rig_omni/
├─ main/
│   ├── application.cc/.h      # 应用生命周期、状态机、音频和协议调度
│   ├── mcp_server.cc/.h      # MCP 远程工具框架
│   ├── ota.cc/.h             # OTA 更新
│   ├── audio/                 # 音频、唤醒词、降噪处理
│   ├── display/               # LVGL、LCD、EAF 表情显示
│   └─ boards/
│       ├── common/            # 通用板级能力：Wi-Fi、按钮、IMU、摄像头等
│       ├── puppy/             # RIG-Puppy 专属实现
│       └─ hover/              # RIG-Hover 专属实现
│           ├── hover_board.cc # Hover 板级总入口
│           ├── xgo.cc/.h      # 舵机/轮电机协议、平衡控制、BLE 协议入口
│           ├── hover_debug_server.cc # Web 调参服务器
│           ├── board_config.h  # Hover 引脚与板级配置
│           ├── emoji/          # EAF 表情资源
│           ├── 240_240/        # 表情布局与资源配置
│           └─ wakenet/         # Hover 专属唤醒词模型
├─ tools/                      # 资源打包、固件包生成脚本
├─ partitions/16m.csv          # 16MB Flash 分区表
└─ sdkconfig.defaults*         # 默认构建配置
```

最值得先读的 4 个文件：

```
main/boards/hover/hover_board.cc
```

```
main/boards/hover/xgo.cc
```

```
main/boards/hover/hover_debug_server.cc
```

```
main/Kconfig.projbuild
```

4. 构建和烧录

准备环境：

ESP-IDF v5.5.3 或更高版本

Python 3.8+

ESP32-S3 开发环境

建议使用 16MB Flash + PSRAM 的 ESP32-S3 模组

拉取代码：

```
git clone https://github.com/LuwuDynamics/rig_omni.git
cd rig_omni
```

加载 ESP-IDF：

```
source ~/esp/esp-idf/export.sh
```

设置目标芯片：

```
idf.py set-target esp32s3
```

选择 RIG-Hover：

```
idf.py menuconfig
```

进入：

```
RIG-Omni -> Board Type -> RIG-Hover
```

也可以直接在 `sdkconfig` 中确认：

```
CONFIG_BOARD_TYPE_HOVER=y
```

构建、烧录和串口监视：

```
idf.py build  
idf.py -p /dev/ttyUSB0 flash monitor
```

Windows 下端口通常类似：

```
idf.py -p COM5 flash monitor
```

生成发布固件包：

```
python tools/gen_bin_package.py
```

RIG-Hover 的输出固件名由 `main/CMakeLists.txt` 设置为：

```
rig-hover
```

5. RIG-Hover 启动链路

RIG-Hover 的板级类是 `HoverBoard`，定义在 `hover_board.cc`，最后通过：

```
DECLARE_BOARD(HoverBoard);
```

注册给通用 `Board` 框架。

构造函数中的初始化顺序大致是：

```
InitializeSpi()
InitializeLcdDisplay()
InitializeButtons()
InitializeTools()
InitializeCamera()
InitializeUart()
InitializeController()
InitializeBootButton()
imu_init()
创建 xgo_control 任务
创建 xgo_rx 任务
创建 imu_read_once 任务
```

这里三个核心 FreeRTOS 任务：

任务	所在线程逻辑	作用
xgo_task	循环调用 xgo_control()	计算平衡控制量并发送轮电机/头部舵机指令
xgo_rx_task	循环调用 xgo_rx()	接收电机反馈，更新轮速、位置、舵机角度
imu_task	循环调用 imu_read_once()	更新 roll、pitch、yaw、角速度等姿态变量

开发时建议先把启动日志跑通，确认这些日志能看到：

```
Initialize UART
XGO control tasks created
IMU control tasks created
Initialization complete
Debug server started at http://<device-ip>
```

6. 运动控制核心

RIG-Hover 的运动控制主要在 `main/boards/hover/xgo.cc` 。

6.1 关键全局变量

变量	含义
<code>vx</code>	目标前进速度
<code>vyaw</code>	目标转向速度，目前主控制中主要使用 <code>stable_yaw</code> 和 <code>yaw_u</code>
<code>target_head_pos</code>	头部舵机目标角度，单位近似为度
<code>wheel1_vel</code> / <code>wheel2_vel</code>	左右轮反馈速度
<code>wheel1_x</code> / <code>wheel2_x</code>	左右轮累计位置
<code>wheel_x</code>	机器人前后位置估计
<code>wheel_vx</code>	平均轮速，变量名里有拼写遗留
<code>pitch</code> / <code>roll</code> / <code>yaw</code>	IMU 姿态反馈，来自 <code>imu.h</code>
<code>dq</code>	pitch 角速度反馈
<code>stable_pos</code>	站稳时的目标位置
<code>stable_yaw</code>	站稳时的目标航向
<code>imu_zero</code>	平衡零点角度补偿
<code>robot_state</code>	0 表示停止输出力矩，1 表示允许平衡控制
<code>lqr_k[4]</code>	LQR 全状态反馈参数

6.2 控制器初始化

`InitializeController()` 设置默认参数：

```
pid_pos.fpKp = 60.0;
pid_vel.fpKp = 0.15;
pid_vel.fpKi = 0.01;
pid_pit.fpKp = 21.0;

kd_pit = 2.3;
imu_zero = -5.0;

lqr_k[0] = 1400.0f;
lqr_k[1] = 6.8f;
```

```
lqr_k[2] = 32.0f;  
lqr_k[3] = 1.6f;
```

当前主控制逻辑已经从传统串级 PID 迁移到离散 LQR。文件里保留了位置、速度、俯仰 PID 的旧逻辑注释，适合学习对比。

6.3 状态更新

`update_state()` 做几件事：

根据左右轮累计位置计算 `wheel_x`

根据左右轮速度计算平均速度 `wheel_vx`

判断机器人是否已经稳定站立

姿态超过安全范围时关闭力矩输出

轮速过高时判断为被拿起或异常状态

关键保护条件：

```
pitch >= 60 deg 或 roll >= 40 deg 持续 500 ms -> robot_state = 0  
pitch < 7 deg 且 roll < 7 deg 持续 1500 ms -> robot_state = 1  
轮速绝对值 > 350 持续多次 -> robot_state = 0
```

这部分是二次开发必须保留的安全边界。调参时建议先让保护逻辑稳定工作，再提高控制输出。

6.4 LQR 控制流程

`xgo_control()` 的核心状态量：

```
pitch_ref = imu_zero * cosf(q_head * PI / 180.0f);  
lqr_x = wheel_x - stable_pos;  
lqr_vx = wheel_vx - vx;  
lqr_q = pitch - pitch_ref;  
lqr_dq = dq;  
  
temp_u = -(lqr_k[0] * lqr_x  
           + lqr_k[1] * lqr_vx  
           + lqr_k[2] * lqr_q  
           + lqr_k[3] * lqr_dq);
```

随后做输出限制：

```
temp_u = Clip(temp_u, -pid_pit.fpUMax, pid_pit.fpUMax);
```

航向控制：

```
yaw_u = k_yaw * (yaw - q_head - stable_yaw);  
yaw_u = Clip(yaw_u, -80, 80);
```

左右轮力矩输出：

```
tor1 = -temp_u + yaw_u;  
tor2 = temp_u + yaw_u;
```

如果 `robot_state == 0`，左右轮力矩直接清零。

6.5 舵机和轮电机协议

Hover 使用 UART2，波特率 1 Mbps：

```
uart_driver_install(UART_NUM_2, 1024, 1024, 0, NULL, 0);  
uart_param_config(UART_NUM_2, &uart_cfg);  
uart_set_pin(UART_NUM_2, XGO_UART_TX_PIN, XGO_UART_RX_PIN, ...);
```

头部舵机控制：

```
WriteByte_P_V(3, short(1500.0 - target_head_pos * 10.0), 1300);
```

轮电机力矩通过 `sendWheelTor()` 下发，电机 ID 为 1 和 2：

```
torque[1] = tor1;  
torque[2] = tor2;
```

7. MCP 远程控制接口

RIG-Hover 在 `hover_board.cc` 中注册了三个机器人控制工具：

MCP 工具名	参数	功能
<code>self.robot.head_angle</code>	<code>angle: -45~45</code>	设置头部舵机角度
<code>self.robot.move</code>	<code>distance: -20~20</code>	修改 <code>stable_pos</code> ，实现前进/后退
<code>self.robot.rotate</code>	<code>angle: -180~180</code>	修改 <code>stable_yaw</code> ，实现原地转向

示例逻辑：

```
target_head_pos = angle;
stable_pos = stable_pos + distance;
stable_yaw = stable_yaw + angle;
```

开发建议：

- 新增动作时优先注册 MCP 工具，而不是写死在控制循环中。
- 工具回调中只修改目标量，不要做长时间阻塞。
- 运动幅度先限制小范围，确认平衡稳定后再放宽。

8. Web 调试服务器

RIG-Hover 有专属调试服务器： `hover_debug_server.cc` 。

启动条件：

- 设备 Wi-Fi 已连接
- `OnInitializationComplete()` 或 `OnWifiConfigEnd()` 中调用 `hover_debug_server_start()`

浏览器访问：

```
http://<RIG-Hover-IP>/
```

页面功能：

- 实时查看 IMU 的 roll、pitch、yaw
- 通过 `/api/set` 在线调整控制参数

每 500 ms 自动刷新姿态数据

接口：

```
GET /api/data
GET /api/set?i=<index>&v=<value>
```

调试变量映射：

index	页面标签	对应变量	建议用途
0	head	target_head_pos = value	测试头部舵机
1	delta_pos	stable_pos += value	小步前进/后退
2	POS_kp	pid_pos.fpKp = value	旧 PID 参数，当前主控不直接使用
3	POS_kd	pid_pos.fpKd = value	旧 PID 参数
4	VEL_kp	pid_vel.fpKp = value	旧 PID 参数
5	VEL_ki	pid_vel.fpKi = value	旧 PID 参数
6	PIT_kp	pid_pit.fpKp = value	当前主要用于输出上限结构体字段，保留调试意义
7	PIT_kd	kd_pit = value	旧 PID 微分项
8	YAW_kp	实际代码映射为 imu_zero = value	页面标签与代码含义不一致，调试时要注意
9	delta_yaw	stable_yaw += value	小角度转向
10	LQR_k0	lqr_k[0] = value	位置误差反馈
11	LQR_k1	lqr_k[1] = value	速度误差反馈
12	LQR_k2	lqr_k[2] = value	俯仰角误差反馈
13	LQR_k3	lqr_k[3] = value	俯仰角速度反馈

注意：页面显示 YAW_kp ，但代码里 index 8 设置的是 imu_zero ；真正的 k_yaw 当前没有被 Web 页面修改。这个地方建议后续修正文案或增加单独的 k_yaw 调参入口。

9. 推荐学习路线

第 1 天：跑通工程

目标：固件能编译、烧录、串口有日志。

阅读：

README.md

README_CN.md

main/Kconfig.projbuild

main/CMakeLists.txt 中 CONFIG_BOARD_TYPE_HOVER 分支

实验：

```
idf.py set-target esp32s3
idf.py menuconfig
idf.py build
idf.py -p <PORT> flash monitor
```

检查点：

串口能看到 HOVER 标签日志

LCD 能播放启动表情

按键/触摸能触发配网或聊天开关

第 2 天：理解板级初始化

目标：知道每个外设在哪里初始化。

阅读：

hover_board.cc

board_config.h

boards/common/wifi_board.cc

boards/common/button.cc

boards/common/imu.*

实验：

修改启动表情： `display_->SetEmotion("launch")`

修改长按 NVS 清除时间： `kLongPressResetMs`

打印 IMU 姿态到日志

第 3 天：理解串口电机协议

目标：知道如何给舵机和轮电机发命令，如何读取反馈。

阅读：

`xgo.h`

`xgo.cc` 中 `SendMotorCommand()`

`ReadWheelState()`

`ReadMotorState()`

`WriteByte_P_V()`

`xgo_rx()`

实验顺序：

只测试头部舵机 ID 3。

读取轮电机 ID 1/2 的速度和位置。

小力矩输出，观察左右轮方向是否一致。

第 4 天：理解平衡控制

目标：知道 Hover 为什么能站住。

阅读：

`InitializeController()`

`update_state()`

`xgo_control()`

理解状态量：

位置误差： `wheel_x - stable_pos`

速度误差： `wheel_vx - vx`

角度误差: $\text{pitch} - \text{pitch_ref}$

角速度误差: dq

实验:

在 Web 页面观察 pitch、roll、yaw

小幅调整 `imu_zero`

小幅调整 `LQR_k2` 和 `LQR_k3`

建议先不要改 `LQR_k0`，位置反馈过大容易让车体出现往返冲击。

第 5 天：开发新动作

目标：通过 MCP 或 Web API 增加一个简单动作。

示例：新增“点头/转头”动作。

在 `InitializeTools()` 中添加：

```
mcp_server.AddTool("self.robot.look_left",
    "让 RIG-Hover 向左看",
    PropertyList(),
    [this](const PropertyList& properties) -> ReturnValue {
        target_head_pos = 30;
        return true;
    });
```

更好的方式是做成带参数工具：

```
mcp_server.AddTool("self.robot.set_motion_target",
    "设置 RIG-Hover 的移动和转向目标",
    PropertyList({
        Property("distance", kPropertyTypeInteger, -20, 20),
        Property("yaw", kPropertyTypeInteger, -90, 90),
    }),
    [this](const PropertyList& properties) -> ReturnValue {
        stable_pos += properties["distance"].value<int>();
        stable_yaw += properties["yaw"].value<int>();
    });
```

```
return true;

});
```

10. 调参建议

调 RIG-Hover 时，要先确认机械、电机方向和 IMU 坐标系，再调参数。

10.1 调试前检查

轮子悬空时，左右轮反馈速度方向是否符合预期。

机身前倾时，`pitch` 的正负方向是否与控制器预期一致。

`target_head_pos = 0` 时，头部是否居中。

`imu_zero` 是否让机器人在自然直立姿态附近平衡。

摔倒保护是否能快速切断力矩。

10.2 参数顺序

推荐顺序：

`imu_zero`

`LQR_k2`

`LQR_k3`

`LQR_k1`

`LQR_k0`

`k_yaw`

含义：

`imu_zero` 决定平衡参考角。

`LQR_k2` 决定对俯仰角偏差的恢复力度。

`LQR_k3` 决定阻尼，太小会摆，太大反应迟钝。

`LQR_k1` 抑制速度偏差。

`LQR_k0` 把机器人拉回目标位置。

`k_yaw` 控制航向保持力度。

10.3 安全范围

建议每次只小幅调整：

```
imu_zero: 每次 0.5 deg
LQR_k2: 每次 1~3
LQR_k3: 每次 0.1~0.3
LQR_k1: 每次 0.5~1
LQR_k0: 每次 50~100
```

如果出现高频抖动，优先降低 `LQR_k2` 或提高一点 `LQR_k3`。如果出现慢速来回游走，检查 `imu_zero`、轮速反馈和 `LQR_k0`。

11. 常见开发任务

11.1 修改 Hover 引脚

改：

```
main/boards/hover/board_config.h
```

重点检查：

- UART TX/RX 是否与电机总线一致
- IMU I2C 是否和摄像头 SCCB 分开
- Touch GPIO 是否被其他外设复用

11.2 新增表情

放入：

```
main/boards/hover/emoji/
```

然后在需要的地方调用：

```
display_->SetEmotion("your_emotion_name");
```

如果是新的 EAF 资源，还要检查：

```
main/boards/hover/240_240/emote.json
```

11.3 修改唤醒词模型

RIG-Hover 当前模型路径：

```
main/boards/hover/wakenet/wn9_xiaolutongxue
```

构建系统中对应：

```
set(WAKENET_MODEL "wn9_xiaolutongxue")
set(WAKENET_SRC "${CMAKE_CURRENT_SOURCE_DIR}/boards/${BOARD_DIR}/wakenet/${WAKENET_MODEL}")
```

11.4 增加 Web 调试变量

改：

```
main/boards/hover/hover_debug_server.cc
```

需要同步修改：

`VAR_LABELS`

`set_handler()` 中的 `switch(index)`

必要时增加 `/api/data` 返回字段

建议把 index 8 的标签从 `YAW_kp` 改成 `imu_zero`，或者把代码改成真正控制 `k_yaw`。

11.5 增加新的运动模式

推荐做法：

在 `xgo.cc` 中新增目标变量，比如 `motion_mode`。

在 MCP 工具里只设置目标变量。

在 `xgo_control()` 中根据 `motion_mode` 改变 `stable_pos`、`stable_yaw` 或 `vx`。

保留 `robot_state == 0` 清零力矩逻辑。

不要在 MCP 回调里长时间 `vTaskDelay()`，否则会影响应用响应。

12. 代码阅读地图

flowchart TD

```
A["app_main / Application"] --> B["Board::GetInstance"]
B --> C["HoverBoard"]
C --> D["InitializeSpi + LCD + EmoteDisplay"]
C --> E["InitializeButtons"]
C --> F["InitializeTools / MCP"]
C --> G["InitializeUart"]
C --> H["InitializeController"]
C --> I["imu_init"]
C --> J["xgo_control task"]
C --> K["xgo_rx task"]
C --> L["imu_read_once task"]
J --> M["update_state"]
M --> N["LQR balance control"]
N --> O["sendWheelTor"]
K --> P["ReadWheelState / ReadMotorState feedback"]
F --> Q["self.robot.move / rotate / head_angle"]
C --> R["hover_debug_server_start"]
```

13. 二次开发注意事项

RIG-Hover 是自平衡机器人，任何电机方向、IMU 方向、零点角度错误都会导致倒车或剧烈抖动。

先做悬空测试，再做手扶测试，最后才放地独立站立。

不要一开始就大幅提高 LQR 参数。

Web 调参服务器默认端口 80，只适合局域网开发调试，不建议直接暴露公网。

`hover_debug_server.cc` 中部分中文注释出现编码异常，不影响编译，但建议统一保存为 UTF-8。

旧 PID 代码被注释保留，学习时很有价值；实际运行主路径以 LQR 为准。

`xgo_control()` 是实时控制路径，避免加入日志刷屏、动态内存分配或耗时网络操作。

运动目标应通过 `stable_pos`、`stable_yaw`、`vx` 等目标量间接影响控制，而不是绕过平衡控制直接写轮电机力矩。

14. 建议补充到仓库的文档

可以新增以下文件：

```
docs/content/RIG-Hover Developer Guide.md
docs/content/RIG-Hover Tuning Guide.md
docs/content/RIG-Hover Hardware Bring-up.md
```

也可以在 README_CN.md 中增加一节：

```
## RIG-Hover 快速开发入口

- 板级入口：`main/boards/hover/hover_board.cc`
- 平衡控制：`main/boards/hover/xgo.cc`
- Web 调参：`main/boards/hover/hover_debug_server.cc`
- 引脚配置：`main/boards/hover/board_config.h`
- 构建选择：`idf.py menuconfig -> RIG-Omni -> Board Type -> RIG-Hover`
```

15. 参考链接

项目仓库：[LuwuDynamics/rig_omni](https://github.com/LuwuDynamics/rig_omni)

Hover 板级入口：[main/boards/hover/hover_board.cc](https://github.com/LuwuDynamics/rig_omni/blob/main/main/boards/hover/hover_board.cc)

Hover 控制代码：[main/boards/hover/xgo.cc](https://github.com/LuwuDynamics/rig_omni/blob/main/main/boards/hover/xgo.cc)

Hover 调试服务器：[main/boards/hover/hover_debug_server.cc](https://github.com/LuwuDynamics/rig_omni/blob/main/main/boards/hover/hover_debug_server.cc)

Hover 引脚配置：[main/boards/hover/board_config.h](https://github.com/LuwuDynamics/rig_omni/blob/main/main/boards/hover/board_config.h)

构建配置：[main/Kconfig.projbuild](https://github.com/LuwuDynamics/rig_omni/blob/main/main/Kconfig.projbuild)

ESP-IDF 文档：[Espressif ESP-IDF Programming Guide](https://docs.espressif.com/en/latest/esp-idf/programming-guide/)

16. 项目信息

Hover代码仓库 (GitHub): https://github.com/LuwuDynamics/rig_omni

陆吾智能代码仓库 (GitHub): <https://github.com/luwudynamics>

文档知识库 (语雀): <https://www.yuque.com/luwudynamics/pet>

包含：详细 API 文档、进阶开发教程。

3D 模型 (MakerWorld): <https://makerworld.com.cn/zh/@LuwuDynamics>

包含：全套结构件 STL/3MF 文件，适配拓竹打印配置。

现在就开始打造属于你的 AI 机器人吧！ \ 只需一台3D打印机，就能拥有一个会听、会动、有“灵魂”的伙伴。

技术支持：如有深度开发需求，欢迎加入官方开发者微信群

